

Procesos, Hilos y Planificación

Puntos totales 17/20 ?

Cuestionario de clase.

El objetivo de estos

cuestionarios no es poner una nota sino tener feedback de cómo van siguiendo los temas, así también como a modo de repaso o autoevaluación para cada clase.

No tengan miedo a equivocarse! Cada pregunta tipo multiple choice les dará luego la respuesta junto a una explicación cuando terminen el cuestionario.

✓ ¿Qué partes de la imagen de un proceso deben estar siempre en RAM? * 1/1

- ☐ Stack
- ☐ Heap
- ☐ Código
- ☐ Datos
- ☒ PCB
- ☐ Todo debe estar siempre en memoria
- ☐ Ninguno necesita estar siempre en memoria



Comentarios

El PCB siempre tiene que estar en memoria ya que es la estructura que usa el Sistema Operativo para administrarlo en todo momento.

El resto de la imagen puede pasarse a swap (suspender el proceso) sin ningún problema a fin de liberar memoria para otros procesos. Más adelante en la materia veremos que vamos a poder mandar a swap "de a partes" y no va a ser necesario suspender todo el proceso.



✓ Si un proceso A ejecuta `fork()` creando un proceso B, inmediatamente *1/1
luego de la llamada lo único que cambiará en ambas imágenes es el PID
(process ID)

☐ Verdadero

☒ Falso



Comentarios

También cambiará el resultado de la llamada a `fork()` (guardado en una variable, por lo que cambiaría en el stack si la misma es local o en el segmento de datos si la misma es global). Esto va a permitir diferenciar si quien ejecuta es el proceso padre o el hijo.



✗ ¿Cuáles de las siguientes afirmaciones son correctas? *

0/1

- ☐ Si ocurre un cambio de proceso => va a ocurrir más de un cambio de contexto
- ☐ Si ocurre un cambio de proceso => va a ocurrir más de un cambio de modo
- ☐ Si ocurre un cambio de contexto => va a ocurrir un cambio de proceso
- ☐ Si ocurre un cambio de modo => va a ocurrir un cambio de proceso
- ☒ Si ocurre un cambio de modo => va a ocurrir un cambio de contexto
- ☐ Si ocurre un cambio de contexto => va a ocurrir un cambio de modo



Respuesta correcta

- ☒ Si ocurre un cambio de proceso => va a ocurrir más de un cambio de contexto
- ☒ Si ocurre un cambio de proceso => va a ocurrir más de un cambio de modo
- ☒ Si ocurre un cambio de modo => va a ocurrir un cambio de contexto

Comentarios

Al cambiar de proceso, en el medio tiene que ejecutar el planificador de corto plazo, esto quiere decir que va a pasar de ejecutar un proceso de usuario, luego al SO y luego a otro proceso de usuario. Para que esto ocurra se tiene que pasar de modo usuario -> kernel y luego de kernel -> usuario. En cada uno de estos también se estaría cambiando el contexto de ejecución de la CPU.

Por otro lado, no es cierto que:

- Al cambiar de contexto se cambie de modo.

Por ejemplo cuando uno está atendiendo una interrupción y llega una nueva más prioritaria que se decide atender en forma anidada, se cambia el contexto pero el modo ya se encuentra en modo kernel.

- Al cambiar de contexto o de modo se cambia de proceso.

Por ejemplo un proceso podría ejecutar una syscall, en este caso se cambia de contexto y de modo pero no necesariamente se cambia de proceso, otro ejemplo es que el SO podría atender una interrupción y luego reanudar la ejecución del proceso interrumpido.



✓ ¿Cuáles de las siguientes afirmaciones sobre procesos son Falsas? * 1/1

- ☐ Al finalizar, se liberan los recursos que tenía asignados
- ☒ Por default comparten memoria con otros procesos para poder comunicarse ✓
- ☒ Por default comparten memoria con su proceso padre para poder comunicarse ✓
- ☒ Es la mínima unidad de planificación para el SO ✓
- ☐ Posee un PCB que siempre debe estar en RAM
- ☐ Pueden comunicarse con otros procesos con paso de mensajes
- ☒ Son menos estables y seguros que los hilos (KLTs) ✓
- ☐ Ninguna

Comentarios

A diferencia de los hilos (KLTs), los procesos son independientes entre sí, por lo que no comparten memoria en forma inherente. Para poder comunicarse deben establecer una sección de memoria compartida (a través de una syscall) o con paso de mensajes.

Al finalizar un proceso, el SO libera sus recursos (cosa que no ocurre con los KLTs).

El SO administra todos los procesos del sistema a través de los PCB, por lo que es necesario que por más que un proceso esté suspendido, dicha estructura esté en memoria.

Por último, por el aislamiento que tienen son más estables y seguros que los KLTs (del mismo proceso).



✓ ¿Cuáles de los siguientes planificadores impacta o modifica con sus acciones el grado de multiprogramación? *1/1

☐ Planificador de corto plazo

☒ Planificador de mediano plazo



☒ Planificador de largo plazo



Comentarios

El planificador de largo plazo al admitir nuevos procesos en el sistema aumenta el grado de multiprogramación. Por otro lado, el de mediano plazo al decidir qué procesos manda a swap o vuelve a cargar a memoria lo decrementa o incrementa.

✓ ¿Cuál es el planificador que es más importante que tenga menos overhead? *1/1

☐ El encargado de admitir nuevos procesos al sistema

☐ El encargado de hacer swapping

☒ El encargado de poner procesos en ejecución



Comentarios

El planificador de corto plazo, el encargado de seleccionar procesos para ejecutar, es el que va a ejecutarse muy seguido por lo que es necesario que tome buenas decisiones y que tenga el menor overhead posible.



✓ ¿Qué es el tiempo de espera? *

1/1

- ☐ El tiempo en que el proceso está en la cola de bloqueado
- ☐ El tiempo en que el proceso no está en ejecución
- ☒ El tiempo en que el proceso está en la cola de listos
- ☐ El tiempo en que el proceso está suspendido



Comentarios

Es el tiempo que le negamos CPU al proceso, podríamos haberlo elegido para ejecutar por estar en la cola de listos, pero el planificador seleccionó a otro.

✓ ¿Cuál/es de las siguientes afirmaciones son falsas sobre FIFO? *

1/1

- ☐ Podría permitir que un proceso monopolice la CPU
- ☐ Podría ser útil para correr procesos secuenciales
- ☐ Minimiza los cambios de contexto
- ☐ Todas
- ☒ Ninguna



Comentarios

*Todas las afirmaciones son correctas.
Al no tener desalojo, se espera a que el proceso voluntariamente libere la CPU, esto potencialmente podría generar que un proceso nunca la libere.
En un contexto en el que se quiere correr un lote de procesos minimizando el overhead, es una buena opción.
Al ejecutar un proceso después del otro minimiza la replanificación y por ende los cambios de contexto.*



✓ Indique cuál de las siguientes afirmaciones son correctas sobre SJF * 1/1

- ☒ Puede implementarse con o sin desalojo ✓
- ☒ Minimiza el tiempo de espera promedio ✓
- ☐ Para poder optimizarlo se puede utilizar un promedio ponderado (media exponencial)
- ☐ Prioriza a los procesos CPU Bound
- ☐ Es un algoritmo con poco overhead

Comentarios

-En la versión con desalojo también se le llama SRT (shortest remaining time) ya que elige al que menos le queda por ejecutar de su próxima ráfaga.

-Al anteponer los procesos cortos a los largos hace que el tiempo de espera promedio se minimice

-Es necesario estimar las siguientes ráfagas para poder implementarlo ya que uno no podría predecir las siguientes ráfagas, para lo cual se puede usar una fórmula de promedio ponderado.

-Prioriza a los procesos IO Bound.

-Tiene bastante overhead por el cálculo, aún más si es con desalojo.

✓ HRRN podría implementarse con desalojo sin mayores desventajas * 1/1

- ☐ Verdadero
- ☒ Falso ✓

Comentarios

Debido al cálculo del response ratio, que tiene en cuenta la variable "tiempo de espera" dentro de su fórmula, en caso de implementarlo con desalojo habría que recalcularlo para todos los procesos listos ante cada evento de replanificación, generando muchísimo overhead.



✗ En RR el SO lanza una interrupción para desalojar al proceso en ejecución *0/1 y selecciona al siguiente proceso en LISTO

☒ Verdadero

✗

☐ Falso

Respuesta correcta

☒ Falso

Comentarios

No es el SO el que interrumpe la ejecución sino el timer (que es programado por el planificador antes de poner a ejecutar el proceso de usuario) lanzando una interrupción de fin de quantum. Luego el SO atiende dicha interrupción y el planificador selecciona a otro proceso para ejecutar.

✓ Para un algoritmo de tipo "Feedback" no es suficiente saber que tiene dos colas de planificación y que ambas utilizan RR para poder implementar el algoritmo. *1/1

☒ Verdadero

✓

☐ Falso

Comentarios

Es necesario saber también:
- A qué cola ingresan los procesos nuevos?
- Cuál es el algoritmo entre colas?
- Hay desalojo entre ellas?
- Cuál es el criterio para pasar de una cola a otra?



✓ ¿Cuáles de los siguientes algoritmos podrían sufrir de inanición? *

1/1

☐ FIFO

☒ SJF



☒ Por prioridades



☐ RR

☐ VRR

☐ HRRN

☒ Feedback



Comentarios

FIFO: respeta el orden de llegada, por lo que atenderá a todos en dicho orden.

SJF: al priorizar a los procesos de ráfagas cortas puede ocurrir que un proceso largo nunca sea elegido para ejecutar.

Por prioridades: sucede lo mismo que en SJF, pero teniendo en cuenta la prioridad.

RR: igual que FIFO, pero con límite de Q.

VRR: podría generar la impresión de generar inanición por la cola prioritaria. Sin embargo, lo que hay que recordar es que ese Q' tiende a 0, es decir, cuando finalmente consuma todo el Q inicial vuelve a la cola menos prioritaria.

HRRN: fuera de que privilegia a los procesos de ráfagas cortas, al meter en la ecuación el tiempo de espera hace que los procesos largos no sufran de inanición.

Feedback: depende mucho de la configuración, pero al tener varias colas de atención de distintas prioridades, puede ocurrir inanición tanto en una cola en particular como entre ellas dependiendo de los algoritmos utilizados y de los criterios para subir/bajar de prioridad.



✓ Tanto los procesos como los KLTs y los ULTs son creados a través de syscalls brindadas por el SO *1/1

☐ Verdadero

☒ Falso



Comentarios

Los procesos y los KLTs son creados a través de syscalls. Sin embargo, como el SO no conoce ni "ve" a los ULTs no los crea, sino que esto se realiza a través de bibliotecas de ULTs (que ejecutan a nivel usuario).

✓ La creación y switcheo entre ULTs del mismo KLT/proceso son más livianos que entre KLTs de un mismo proceso. *1/1

☒ Verdadero

☐ Falso



Comentarios

Todo ese manejo (creación y cambio de uno a otro) se realiza en modo usuario sin tener que hacer cambios de modo (el SO no interviene).



✓ En un SO que soporta el manejo de hilos. ¿Qué planificará el planificador *1/1 de corto plazo?

☐ Procesos

☒ KLTs



☐ ULTs

Comentarios

En caso de que el SO soporte la creación y administración de hilos (KLTs), ésta será la unidad de planificación de corto plazo. Por otro lado, nunca verá a los ULTs por lo que no podrá planificarlos.

Esto no quiere decir que el SO no lleve cuenta del estado de cada proceso en su PCB, pero a la hora de elegir qué ejecutar se elige con más granularidad (por KLT en vez de por Proceso)

✓ ¿En caso de utilizar jacketing sería lo mismo utilizar ULTs que KLTs? * 1/1

☐ Sí

☒ No



Comentarios

El SO sigue sin "ver" dichos ULTs por lo que no puede planificarlos (ej, asignarles un Q, o ejecutarlos en distintas CPUs)

Recordar que lo único que soluciona jacketing es que el KLT/Proceso asociado a los ULTs no se bloquee y la biblioteca pueda planificar otro ULT en caso de tener alguno listo.



✓ Indique cuáles de las siguientes afirmaciones son verdaderas sobre planificación con biblioteca de ULTs *1/1

- ☐ Si hay varios KLTs con ULTs todos los KLTs deben usar la misma biblioteca de ULTs.
- ☐ Si decidimos usar ULTs es igual de útil usar directo syscalls (ej write) que usar los wrappers de las syscalls provistas por la biblioteca.
- ☐ Cuando un ULT necesita realizar una IO se lo pide al SO con una syscall. En este caso, el SO puede diferenciar cuál ULT lo está pidiendo.
- ☐ Todas son correctas
- ☒ Ninguna es correcta ✓

Comentarios

Son todas falsas:

- Cada KLT puede usar una biblioteca distinta, y por ende tener distintos criterios de planificación.
- Si llamamos directo a syscalls bloqueantes desde el código de nuestros ULTs no permitimos que la biblioteca replanifique cuando sea necesario para el algoritmo usado.
- El SO no "ve" ni sabe de la existencia de los ULTs, cuando le llega el pedido de una syscall solamente sabe que vino de una entidad de planificación que conoce, como un KLT o un proceso.



✓ Un proceso posee 3 KLTs y cada uno 3 ULTs asociados. Si un ULT necesita hacer una syscall bloqueante (ej read) ... ¿Cuáles de las siguientes afirmaciones PODRÍAN ser ciertas? *1/1

- ☒ Por default se bloquea el KLT ✓
- ☒ Podría bloquearse el KLT y también el proceso asociado ✓
- ☒ Podría no bloquearse el KLT ✓
- ☒ Al finalizar la operación bloqueante y ser seleccionados nuevamente por el planificador del SO, continúa ejecutando el mismo ULT ✓
- ☒ Al finalizar la operación bloqueante y ser seleccionados nuevamente por el planificador del SO, continúa ejecutando el ULT que seleccione la biblioteca de ULTs ✓

Comentarios

El SO por default "ve" que la syscall llega del KLT asociado, por lo que lo bloquearía. En caso de utilizar jacketing, al no bloquearse el KLT podría ejecutar otro ULT del mismo.

Al finalizar la operación bloqueante:

- *Si se realizó la syscall directo -> va a continuar con el ULT que hizo la llamada.*
- *Si se realizó la syscall desde una función wrapper de la biblioteca -> la misma podrá seleccionar al siguiente ULT a ejecutar (que podría ser el mismo que originó la llamada)*

En caso de que justo el último KLT de un proceso se bloquee, se consideraría todo el proceso en estado bloqueado.



✗ Considere un sistema en el que en un momento dado tenemos el siguiente estado. Sabiendo que el SO planifica con SJF ($\alpha = 0,5$) y que el estimado anterior de KAA fue 3. Responda cuáles de las siguientes afirmaciones son verdaderas.

*0/1

[illegible]

- ☐ El próximo estimado de KAA será 4
- ☐ KAA nunca llega a bloquearse
- ☒ KAA utiliza una biblioteca de ULTs con jacketing
- ☐ La biblioteca de ULTs de KAA utiliza SJF

Respuesta correcta

- ✓ El próximo estimado de KAA será 4
- ✓ KAA utiliza una biblioteca de ULTs con jacketing

Comentarios

- Como el estimado anterior fue de 3 y ejecutó 5 unidades hasta bloquearse, su próximo estimado es de $(3+5)/2$
- Hay un momento en el que todos los ULTs están bloqueados, por lo que el KLT se bloquea (la biblioteca no tiene nada más para ejecutar)
- La biblioteca de KAA usa jacketing ya que vemos que hay un ULT ejecutando mientras otro realiza una IO
- No podemos afirmar que la biblioteca de KAA utilice SJF ya que dependería un poco de los estimados, bien podría ser FIFO

Qué dudas te quedaron o qué conceptos no están claros sobre los temas vistos?



Cómo te vienen pareciendo las clases?

Dejanos los puntos positivos que quieras destacar y propuestas sobre puntos a mejorar.

.....

Les dejamos un espacio para que puedan explayar cualquier otro comentario que les haya quedado.

.....

Este formulario se creó en UTN.BA. - [Contactar con el propietario del formulario](#)

¿Parece sospechoso este formulario? [Informe](#)

Google Formularios



